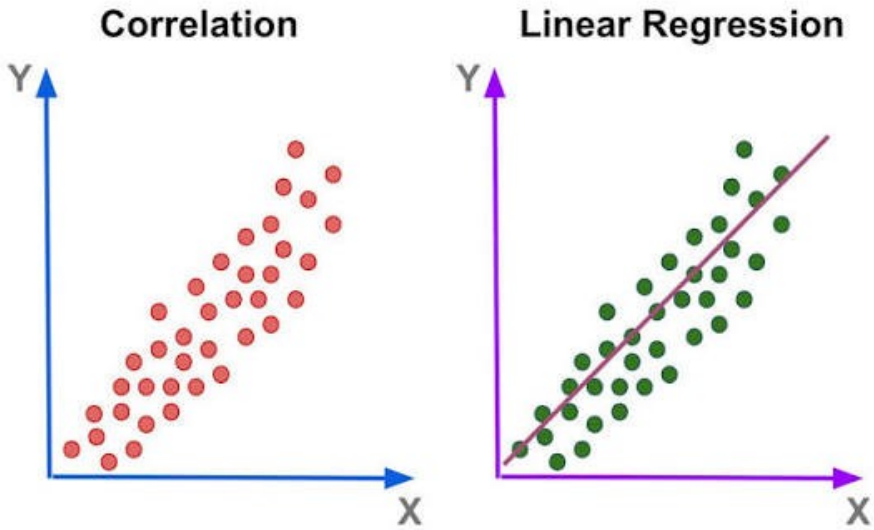
Lecture 26

Prediction II



Announcements

- Course project directed questions are out!

- Office hours every Tuesday (noon to 4pm) and Friday (11am to 1pm)
  - Link to book appointments is on the course website

Today

Linear regression

Delivery Robots

Suppose the dining halls ran a trial using delivery robots on campus

- Tracked delivery time, distance, items damaged, and snow depth
- Trial lasted for 300 deliveries



Using data from the trial, we're asked to

1) Predict how long a delivery will take
2) Predict whether or not items will be damaged

Language of Prediction

Variables

- X denotes inputs X = { distance, snow depth }
- Y denotes output

Goal: predict Y using X

- If Y is numerical, this is called **regression** Y = delivery time
- If Y is categorical, this is called **classification** Y = damaged items

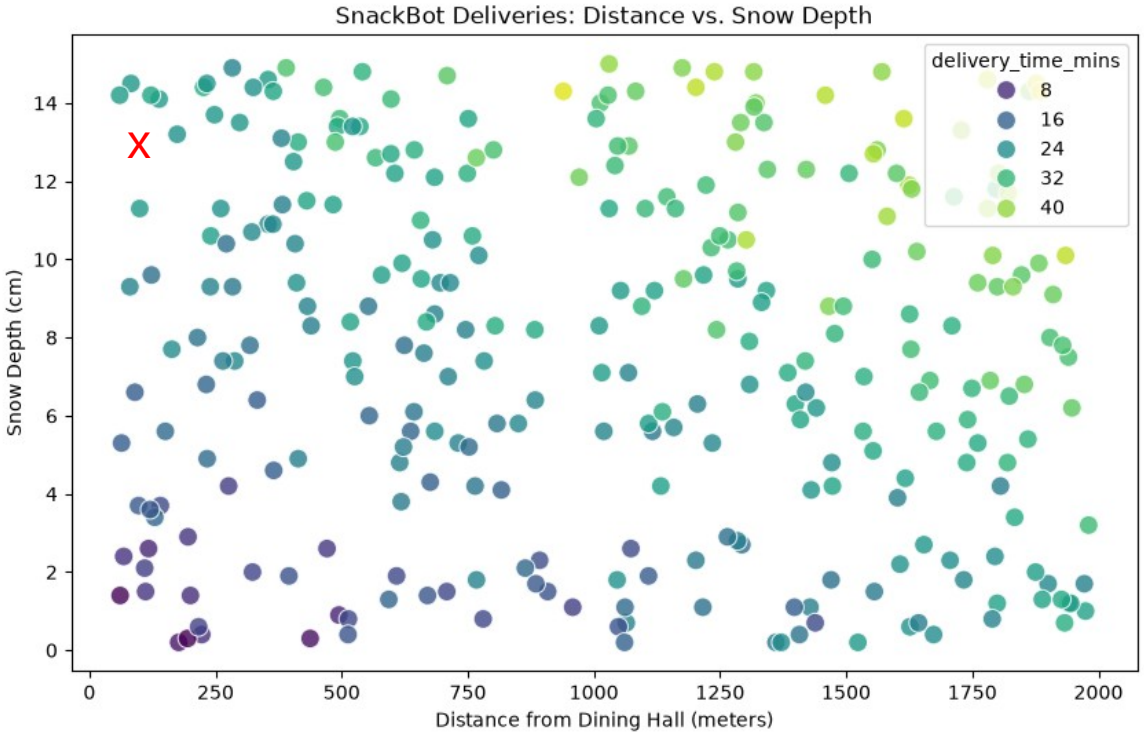
Often call X the **feature space** and individual inputs as **features**

Nearest Neighbors (regression)

Let x be a new input value and let's predict its output

Idea: look at the Y values around x

1) Find the *k* nearest input values to x
2) Get each of their Y values
3) Average them together and output

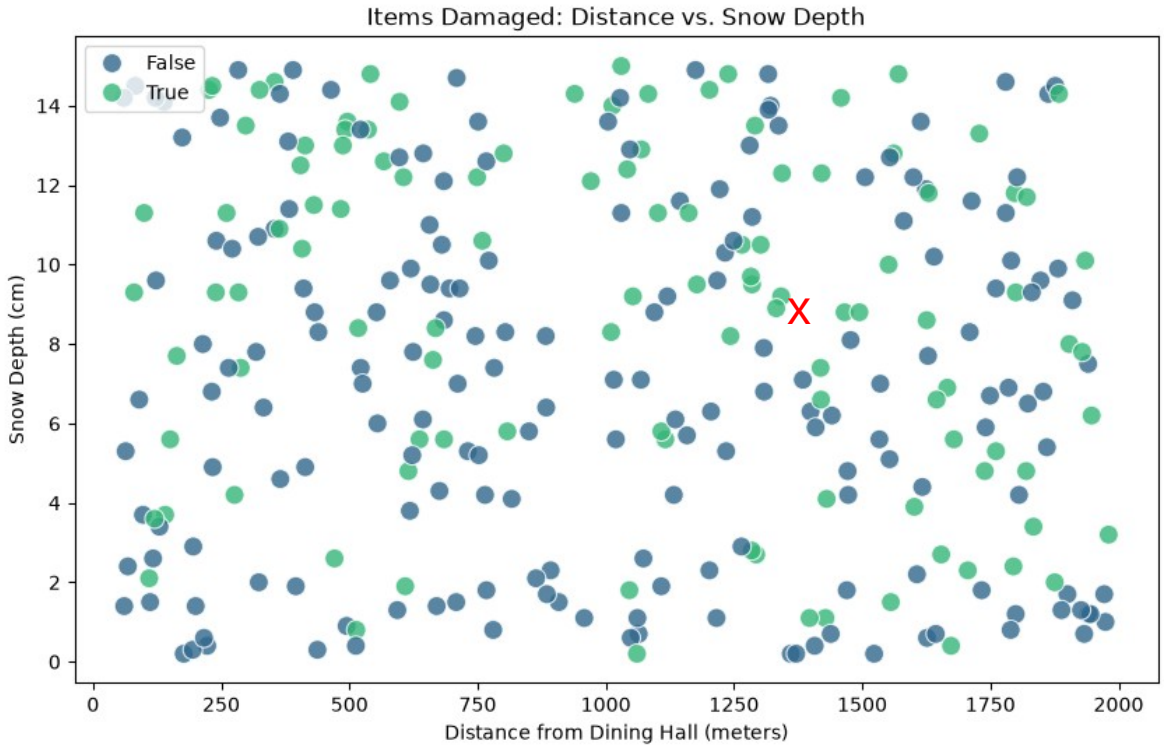


Nearest Neighbors (classification)

Let x be a new input value and let's predict its output

Idea: look at the Y values around x

1) Find the *k* nearest input values to x
2) Get each of their Y values
3) Output value with the most "votes"



Distance in Feature Space

We often *normalize* the features to be between 0 and 1

Distance in Feature Space

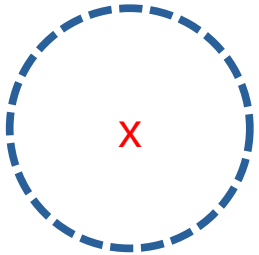
We often *normalize* the features to be between 0 and 1

x

Given a distance, let's look at the points that are *m* away from a point x

Distance in Feature Space

We often *normalize* the features to be between 0 and 1

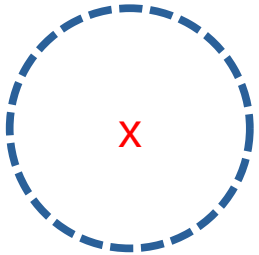


Given a distance, let's look at the points that are *m* away from a point x

- Straight-line/Euclidean distance (L_2)

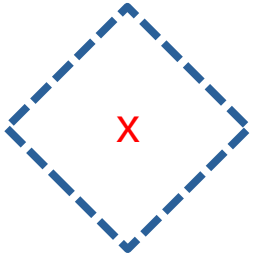
Distance in Feature Space

We often *normalize* the features to be between 0 and 1



Given a distance, let's look at the points that are *m* away from a point x

- Straight-line/Euclidean distance (L_2)
- Coordinate-wise/Taxicab/Manhattan distance (L_1)

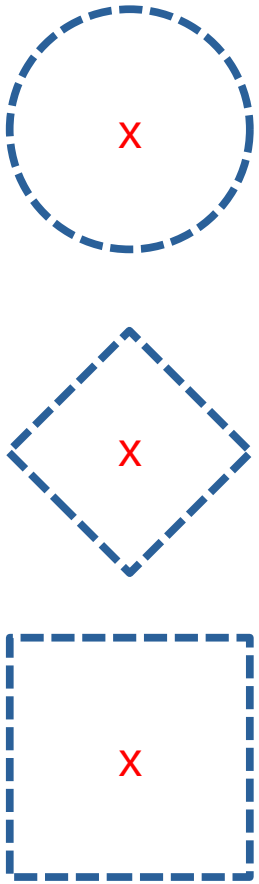


Distance in Feature Space

We often *normalize* the features to be between 0 and 1

Given a distance, let's look at the points that are *m* away from a point x

- Straight-line/Euclidean distance (L_2)
- Coordinate-wise/Taxicab/Manhattan distance (L_1)
- Max distance (L-infinity)



Downsides of NN

Computationally intensive

- Each time you want a prediction, you have to find that point's nearest neighbors
- Not a trivial problem (you'll explore why in CMPSC 202: Algorithms)

Requires us to keep all of the data

- Privacy/security issues
- Storage

Downsides of NN

Computationally intensive

- Each time you want a prediction, you have to find that point's nearest neighbors
- Not a trivial problem (you'll explore why in CMPSC 202: Algorithms)

Requires us to keep all of the data

- Privacy/security issues
- Storage

Would be nice if we can create a summary of the data to use to make predictions

Predictive Model

Rule mapping inputs to outputs that's *learned* from the data

- Give the model some inputs and get an output
- Example: input distance, snow depth and receive delivery time as output

Can use the model without having the access to the data

- Models are a type of statistic

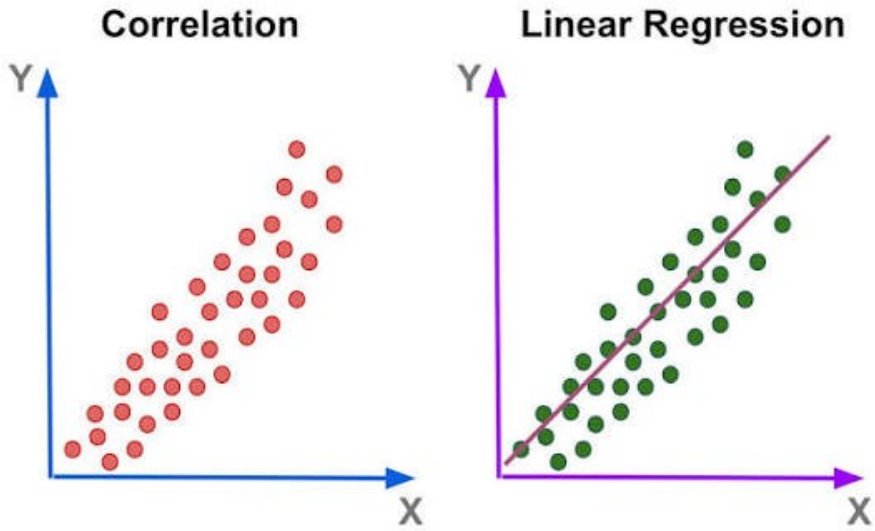
Common model types

- Linear models, trees (e.g., decision trees, random forests), neural networks

Linear Regression

Simple learning method used to predict numerical outputs

- Many machine learning methods are generalizations of linear regression
- Illustrates key concepts while maintaining interpretability



Idea: find the line that minimizes prediction error

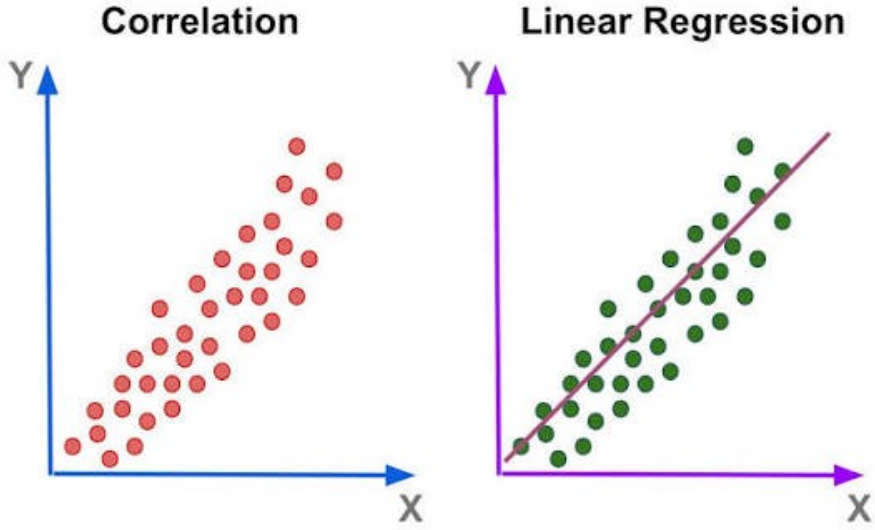
(Simple) Linear Regression

Predict Y on using a single input X

$$Y = \beta_0 + \beta_1 X$$

β_0 and β_1 are the *coefficients* or *parameters* of the model

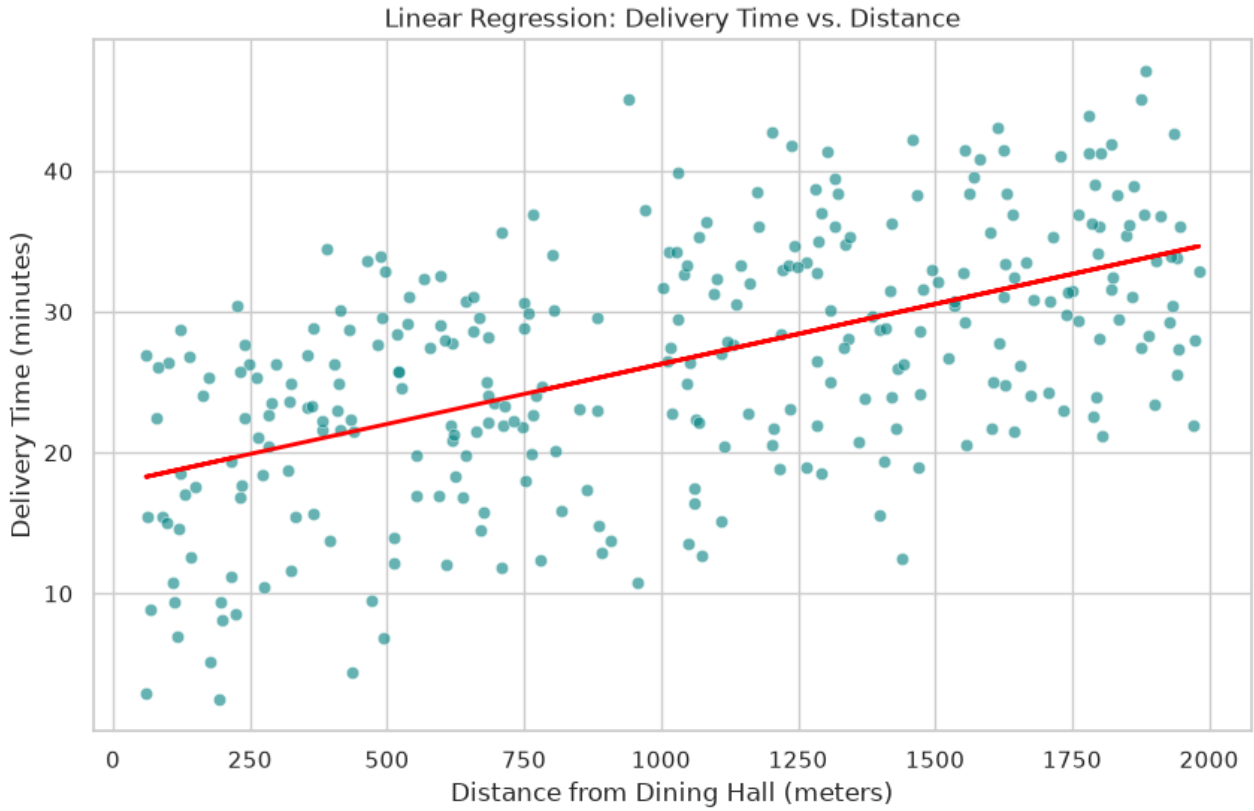
- The intercept and the slope of the line
- Idea: pick coefficients to minimize prediction error



(Simple) Linear Regression

$\beta_0 = 17.77$

$\beta_1 = 0.009$



(Simple) Linear Regression

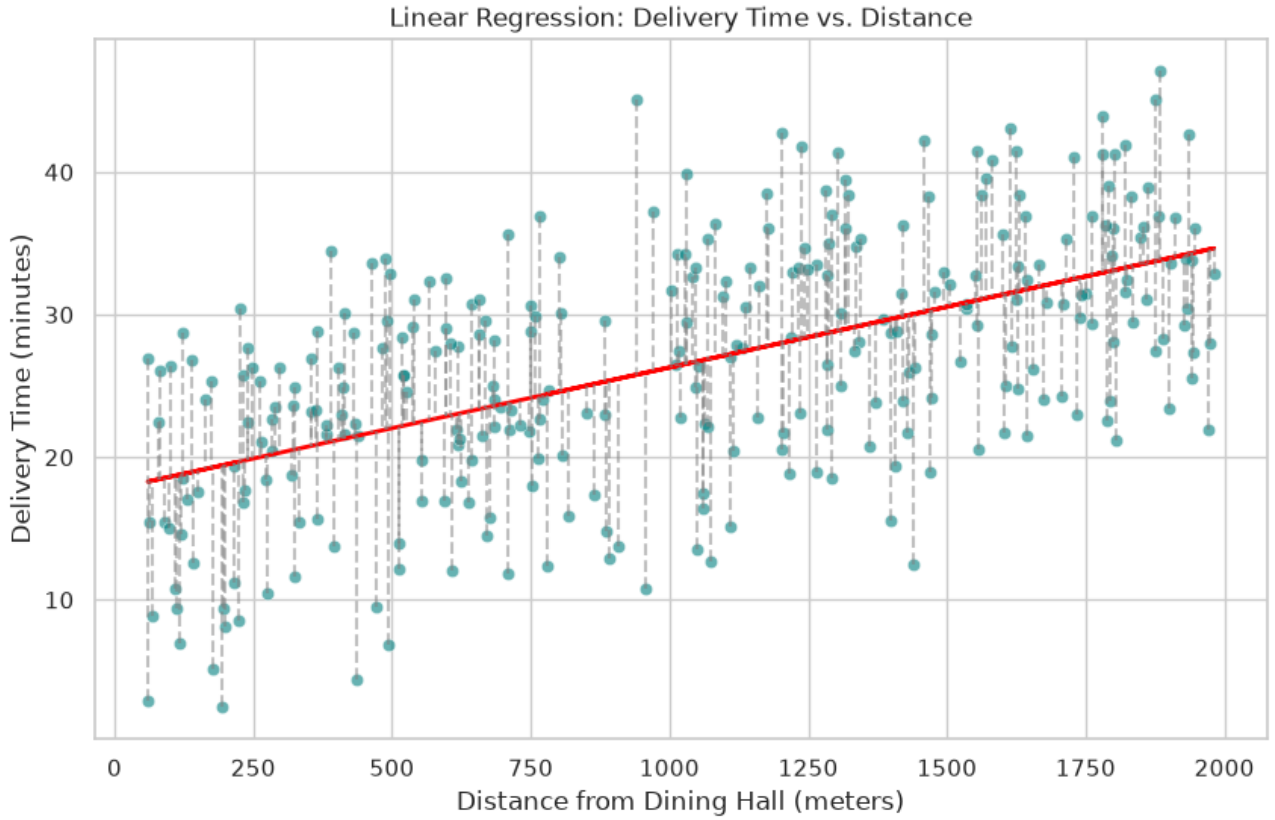
$\beta_0 = 17.77$

$\beta_1 = 0.009$

Dashed lines represent prediction error (residual)

Linear regression finds line that minimizes prediction error

Off by ~6 minutes on average



(Multiple) Linear Regression

What if our dataset contains multiple features?

- We could run a simple linear regression for each input dimension

(Multiple) Linear Regression

What if our dataset contains multiple features?

- We could run a simple linear regression for each input dimension

Predict Y on using inputs $X_1, X_2, \ldots, X_p$

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p$$

(Multiple) Linear Regression

What if our dataset contains multiple features?

- We could run a simple linear regression for each input dimension

Predict Y on using inputs $X_1, X_2, \ldots, X_p$

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p$$

Is this model still a line?

How can we geometrically interpret the model with two input features?

Adding Snow Depth to Regression

Linear model with just distance

- Off by ~6 minutes on average

Linear model with distance and snow depth

- Off by ~2.4 minutes on average

Adding Snow Depth to Regression

Linear model with just distance

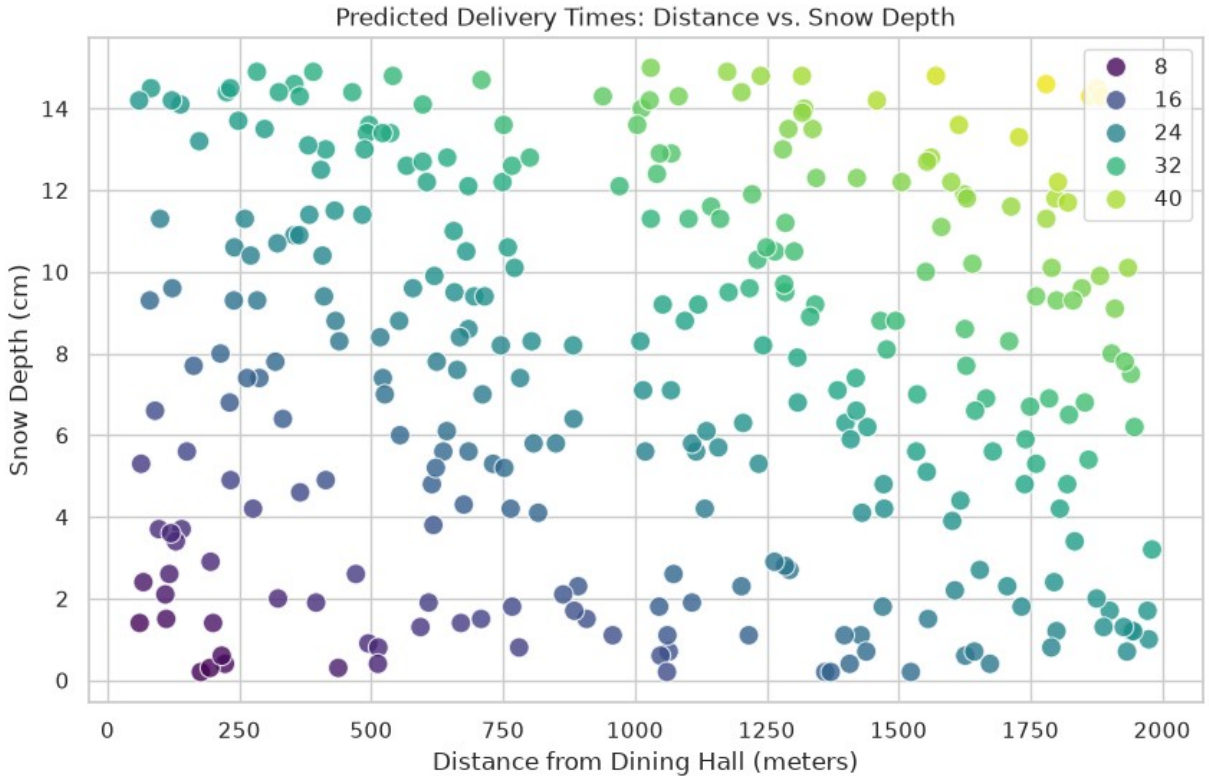
- Off by ~6 minutes on average

Linear model with distance and snow depth

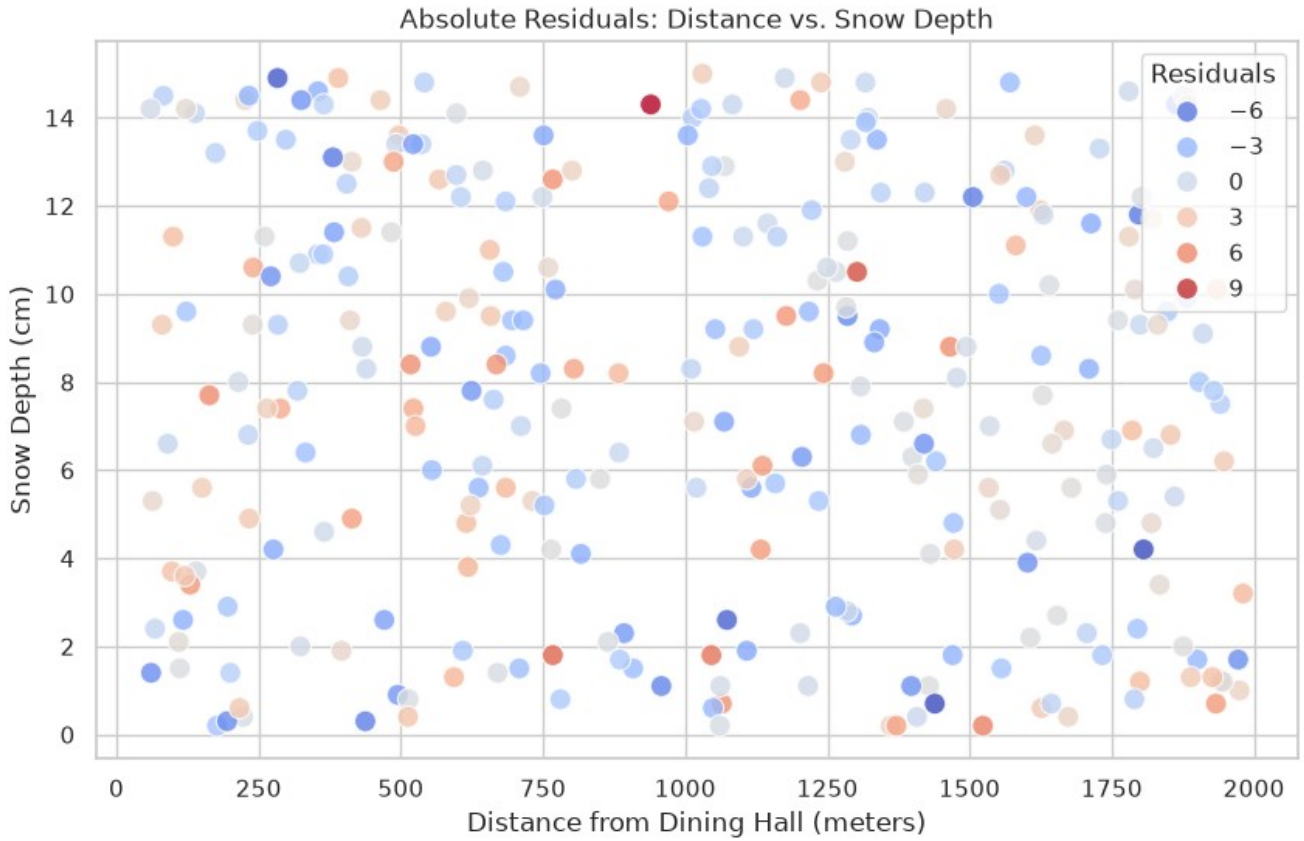
- Off by ~2.4 minutes on average

Why did the second model improve?

Visualizing Multiple Regression



Visualizing Multiple Regression



Next time

- How do we know if a predictive model is doing well?